

06

/ 13 LECTURES

Graph Algorithms for Networking — II

Why Dijkstra breaks on negative edges, how Bellman-Ford fixes it, Minimum Spanning Trees, and the great split that shapes Internet routing: Distance Vector vs Link State.

Bellman-Ford

Negative Edges & Cycles

Kruskal · Prim

Minimum Spanning Tree

STP

Distance Vector

Link State

RIP vs OSPF vs BGP

Autonomous Systems

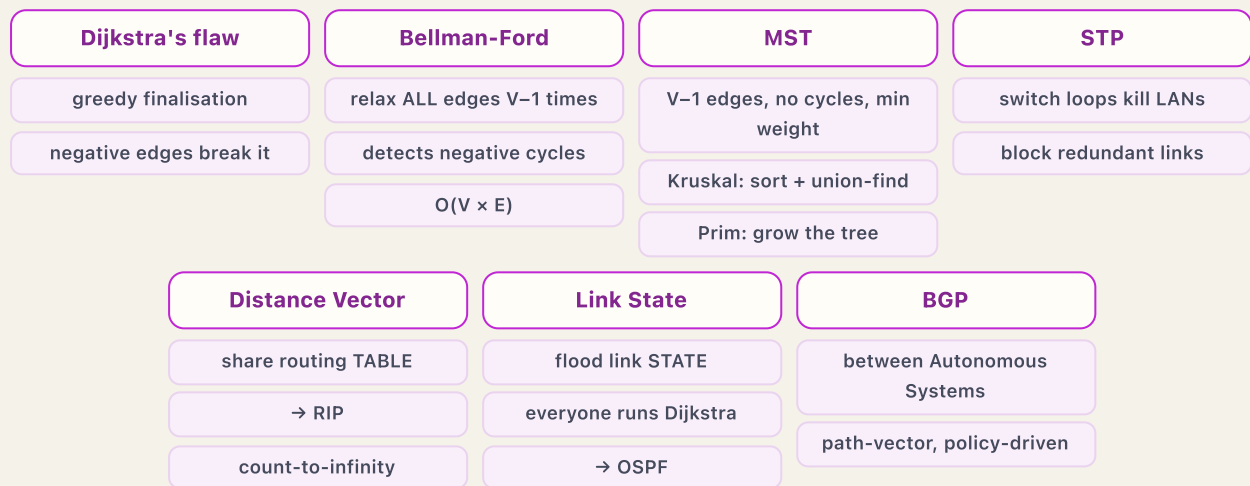
WHAT THIS LECTURE GIVES YOU

The second half of the routing-algorithm story: the counterexample that breaks Dijkstra, Bellman-Ford's V-1 relaxation rounds with a full trace, negative-cycle detection (= routing loops!), MST algorithms and their surprising real use in Ethernet switches (STP), and how the DV-vs-LS choice defines RIP, OSPF and BGP.

LEARNING OBJECTIVES — AFTER THIS LECTURE YOU CAN

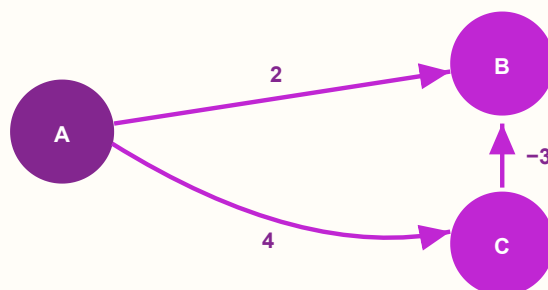
- Explain **why Dijkstra fails** on negative-weight edges (with the counterexample)
- Implement and trace **Bellman-Ford**; explain the V-1 and detect negative cycles
- Define **MST**; trace **Kruskal** and describe **Prim**
- Connect graph algorithms to protocol families: **DV (Bellman-Ford) vs LS (Dijkstra)**

Graphs II



1 Why Dijkstra Fails on Negative Edges

Dijkstra's greedy assumption: **once a node is finalised (popped), its shortest distance is correct**. Negative edges break exactly this.



TRUE shortest A→B:
 $A \rightarrow C \rightarrow B = 4 - 3 = 1$

Dijkstra answers:
 $A \rightarrow B = 2$ ✗ WRONG

it finalises B (2) before
 ever exploring C → B

Figure 1. The classic counterexample — greediness + negative edges = wrong answer.

2 Bellman-Ford — the Fix

Core idea: relax **every edge, V-1 times**. Each full pass propagates shortest paths at least one more hop further. Why V-1? Any shortest path (without a negative cycle) visits at most V-1 edges.

```
def bellman_ford(edges, vertices, source):
    # edges: list of (u, v, weight) — order doesn't matter!
    dist = {v: float('inf') for v in vertices}
    dist[source] = 0
    prev = {v: None for v in vertices}

    for _ in range(len(vertices) - 1):      # V-1 passes
        for u, v, w in edges:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w      # relax
                prev[v] = u

    for u, v, w in edges:                  # V-th pass = detector
        if dist[u] != float('inf') and dist[u] + w < dist[v]:
            raise ValueError("Negative weight cycle detected!")
    return dist, prev
```

Trace — edges (A,B,2), (A,C,4), (C,B,-3), source A

Round	Edge processed	Relaxation	dist after
init	—	—	{A:0, B:∞, C:∞}
1	(A,B,2)	$0+2 = 2 < \infty$ ✓	B=2
	(A,C,4)	$0+4 = 4 < \infty$ ✓	C=4
	(C,B,-3)	$4+(-3) = 1 < 2$ ✓ improves!	B=1
2	all edges	no further improvement	converged: {A:0, B:1, C:4}

Answer: A→B costs **1** via A → C → B — exactly where Dijkstra returned 2.

Negative cycles = routing loops

DETECTED If a **V-th** pass can still relax → a negative cycle exists (e.g. A→B(1)→C(-3)→A(1): each lap -1 → "shortest path" is meaningless). **In networking, a negative cycle IS a routing loop** — packets circulating forever. Bellman-Ford detects it; Dijkstra cannot.

Feature	Dijkstra	Bellman-Ford
Negative edges	✗ fails	✓ handles
Negative cycles	cannot detect	✓ detects
Time complexity	$O((V+E) \log V)$	$O(V \times E)$
Strategy	Greedy	Dynamic programming
Network protocol	OSPF	RIP, BGP (conceptually)
Use when	weights ≥ 0 , performance matters	negative weights / loop detection needed

3 Minimum Spanning Tree (MST)

Spanning tree

Connects **all vertices**, has **no cycles**, exactly **$V-1$** edges.

Minimum spanning tree

The spanning tree with the **lowest total edge weight**.

Two classics

Kruskal: sort edges, add if no cycle (union-find). **Prim**: grow tree from any node, always add cheapest outgoing edge.

KRUSKAL — SORT + UNION-FIND

PYTHON

```
def kruskal(edges, vertices):
    edges = sorted(edges, key=lambda e: e[2])  # 1. sort by weight
    parent = {v: v for v in vertices}

    def find(x):
        # 2. find with path compression
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    mst = []
    for u, v, w in edges:
        ru, rv = find(u), find(v)
        if ru != rv:
            parent[ru] = rv
            mst.append((u, v, w))
    return mst  # stops naturally at V-1 edges
```

Kruskal trace — the Lecture-5 city graph

#	Edge (sorted)	Decision	Tree so far
1	Hyd–Bang (2)	ADD	{Hyd, Bang}
2	Del–Hyd (2)	ADD	{Hyd, Bang, Del}
3	Mum–Hyd (3)	ADD	{Hyd, Bang, Del, Mum}
4	Chen–Bang (4)	ADD — all connected!	{all 5} → stop
5	Mum–Del (5)	SKIP (cycle)	—
6	Del–Chen (8)	SKIP (cycle)	—

MST edges: Hyd–Bang 2, Del–Hyd 2, Mum–Hyd 3, Chen–Bang 4 → **total cost 11** ($4 = V-1$ edges ✓).

MST in real networks — Spanning Tree Protocol (STP, IEEE 802.1D)

The problem

Ethernet switches with **redundant links** create loops — broadcast frames circulate forever (a "broadcast storm") and melt the LAN.

The fix

Switches run a **distributed MST algorithm**: redundant links are **logically blocked**, leaving a loop-free tree. If an active link fails, STP unblocks a spare — automatic failover.

4

Distance Vector vs Link State — the Big Split

Distance Vector (Bellman-Ford based)

- Each router knows costs to **direct neighbours** only
- Periodically broadcasts its **entire routing table** to neighbours
- Update rule: $\text{dist}[\text{dest}] = \min(\text{dist}[\text{dest}], \text{cost_to_nbr} + \text{nbr_dist}[\text{dest}])$
- Converges over many rounds; partial (local) view
- **Protocols:** RIP, BGP (path-vector)

Link State (Dijkstra based)

- Each router floods its **local link state** (neighbours + costs) to ALL routers
- Every router builds the **complete topology map**
- Every router independently runs **Dijkstra** → consistent results
- Fast convergence; no count-to-infinity
- **Protocols:** OSPF, IS-IS

Feature	Distance Vector	Link State
Information shared	Routing table (distances)	Link state (direct neighbours)
Algorithm basis	Bellman-Ford	Dijkstra
Topology knowledge	Partial (local view)	Full (global view)
Convergence speed	Slow	Fast
Bandwidth use	Low (periodic small updates)	High at first (flooding), then quiet
Loop risk	Count-to-infinity	None (consistent full map)
Examples	RIP, BGP	OSPF, IS-IS

EXAM

The one-liner that earns marks: "**DV shares distances, LS shares topology.**" Everything else (convergence speed, loop behaviour, algorithms) follows from that single design difference.

5 How the Internet Stacks Its Routing

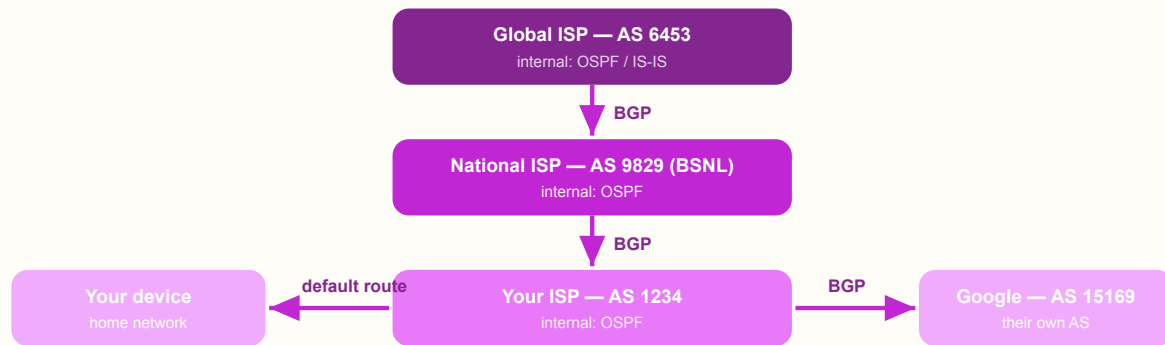


Figure 2. Inside each Autonomous System: OSPF/IS-IS. Between ASes: BGP. Your home router just points a default route at your ISP.

AS An **Autonomous System** = a network under one administrative control with a unique ASN: Google AS15169, Cloudflare AS13335, Jio AS55836, BSNL AS9829. **IGPs** (RIP/OSPF/IS-IS) run *inside*; **BGP** (EGP) runs *between*.

LECTURE 6 — RAPID REVISION CARD

Dijkstra breaks on negative edges — it finalises B(2) before seeing $A \rightarrow C \rightarrow B = 1$.

MST: spans all nodes, $V-1$ edges, no cycles, minimum weight. Kruskal = sort + union-find; Prim = grow.

STP (802.1D): switches build a spanning tree, block redundant links $\rightarrow$ no broadcast storms; failover unblocks spares.

LS shares topology (flood LSAs, everyone runs Dijkstra) $\rightarrow$ OSPF; fast, loop-free.

Bellman-Ford: relax all edges $V-1$ times; V -th pass still relaxes $\rightarrow$ negative cycle. $O(V \times E)$.

City graph MST: Hyd–Bang 2, Del–Hyd 2, Mum–Hyd 3, Chen–Bang 4 $\rightarrow$ **11**.

DV shares distances (table to neighbours, Bellman-Ford) $\rightarrow$ RIP; count-to-infinity risk.

BGP = path-vector between ASes; inside AS = OSPF/IS-IS; home router = default route.

Important Questions — Exam Practice

A. Multiple Choice

Q1. Bellman-Ford relaxes all edges how many times (no negative cycles)?

MCQ

- a) V
- b) $V-1$
- c) E
- d) $\log V$

Q2. An MST of a 10-node connected graph has exactly:

MCQ

- a) 10 edges
- b) 9 edges
- c) 11 edges
- d) depends on weights

Q3. In link-state routing, a router floods:

MCQ

- a) its full routing table
- b) its local link state (neighbours + costs)
- c) only its default route
- d) MAC addresses

Q4. A negative cycle in a routing graph corresponds to:

MCQ

- a) a fast path
- b) a routing loop
- c) a broken cable
- d) an MST

Q5. Which protocol runs BETWEEN Autonomous Systems?

MCQ

- a) OSPF
- b) RIP
- c) BGP
- d) STP

B. Short Answer (2–4 marks)

Q6. Trace Bellman-Ford on $A \rightarrow B(3)$, $B \rightarrow C(-5)$, $A \rightarrow C(2)$ from A. Final distances?

SHORT

Q7. Does a negative edge always imply a negative cycle? Justify.

SHORT

Q8. Why must STP block links? What happens on a switch network without it?

SHORT

Q9. What is the maximum hop count in RIP and why was 16 chosen as "unreachable"?

SHORT

Q10. State the single design difference between DV and LS routing, and one consequence of each.

SHORT

C. Long Answer (5–8 marks)

Q11. With the A/B/C counterexample diagram, explain precisely why Dijkstra's greedy finalisation fails with negative edges, and how Bellman-Ford's repeated relaxation fixes it. Include the full trace.

LONG

Q12. Run Kruskal on Figure 1 of Lecture 5 (city graph): list the sorted edges, the accept/skip decision for each, and the MST total cost. How many edges and why?

NUMERICAL

Q13. Compare Distance Vector and Link State routing (table form): information shared, algorithm, topology view, convergence, loop behaviour, example protocols.

LONG

Q14. Explain the Internet's routing hierarchy with a diagram: what runs inside an AS, what runs between ASes, and what your home router does. Define Autonomous System and give two real ASNs.

LONG

ANSWER KEY

A1. $b - V - 1$ passes; a V -th pass detects negative cycles.

A2. $b - V - 1 = 9$.

A3. b — LSAs describe direct links; tables are what DV shares.

A4. b — cost decreases every lap $\rightarrow$ packets circulate forever.

A5. c — BGP is the EGP; OSPF/RIP are IGPs; STP is Layer 2.

A6. After pass 1: $B=3$, then C : $\min(2, 3-5=-2) = -2 \rightarrow \{A:0, B:3, C:-2\}$; path to C is $A \rightarrow B \rightarrow C$.

A7. No — a single negative edge is fine (see the $A \rightarrow C \rightarrow B$ example); a cycle with negative total is the dangerous case.

A8. Redundant switch links loop broadcasts forever (broadcast storm). STP builds a tree, blocks spares, unblocks on failure.

A9. 15 hops; 16 = infinity/unreachable — bounds count-to-infinity damage and keeps tables small.

A10. DV shares distances (partial view $\rightarrow$ slow convergence, count-to-infinity); LS shares topology (full view $\rightarrow$ fast convergence, no count-to-infinity).

A12. Sorted: $H-B(2)\checkmark$, $D-H(2)\checkmark$, $M-H(3)\checkmark$, $C-B(4)\checkmark$, $M-D(5)\times$ cycle, $D-C(8)\times$ cycle $\rightarrow$ MST cost **11**, 4 edges = $V-1$.

A11/A13/A14. Reproduce Figure 1, the DV-vs-LS table, and Figure 2 — labelled diagrams carry the marks.

Wrap-Up — carry these into Lecture 7

BELLMAN-FORD Relax every edge $V-1$ times $\rightarrow$ handles **negative edges**; a further pass that still improves = **negative cycle**. Dijkstra has no such safety net.

MST Kruskal: sort edges, take the cheapest that doesn't form a cycle (union-find). Prim grows one tree. Both give the same total weight.

DV VS LS DV = "tell your **neighbours** the whole map" (distance vectors). LS = "tell **everyone** your neighbourhood" (link-state floods).

COUNT TO INFINITY Bad news travels slowly in DV — that's why real networks moved to LS (OSPF) inside ASes and patched DV with path vectors (BGP) between them.

NEXT **Lecture 7 — Routing & Forwarding I:** from algorithms to machines — the routing table, longest-prefix match, and how a router moves a packet in nanoseconds.